

Ecosistema de Automatización IA Autónomo

Elaboración de Presupuestos de Rigging de Equipos Médicos
Transporte Sechi

Documentación técnica del ecosistema
23 de octubre de 2026

Índice

1. Introducción
2. Mapa de Arquitectura del Sistema
3. Manual Operativo de Estructuras de Datos
4. Estrategia de Optimización de Costos y Recursos de IA
5. Seguridad, Privacidad y Resiliencia
6. Recursos complementarios

Introducción

Este documento describe el Ecosistema de Automatización IA Autónomo construido para el proceso real de **Elaboración de Presupuestos** de rigging de equipos médicos de alta complejidad en Transporte Sechi.

Desde que un presupuesto entra por un formulario público hasta que el cliente lo recibe por email y queda generado como documento, el ecosistema calcula automáticamente los kilómetros de traslado, identifica la lista de precios aplicable, asigna vehículo y acoplado disponibles, resuelve el precio de rigging y flete, suma recargos y adicionales, redacta el cuerpo del email con un modelo de lenguaje, y genera el documento final del presupuesto sobre la plantilla real de la empresa. Los casos que dependen de criterio humano —necesidad de contratar terceros, visita de acceso, revisión de direcciones o de montos— se pausan explícitamente y retoman solos una vez que una persona confirma la decisión correspondiente.

El ecosistema corre sobre **n8n** como orquestador, con **Airtable** como base de datos relacional del sistema, **Google Gemini** (Flash-Lite) como único modelo de lenguaje del flujo, y **Gmail**, **Slack**, **Google Calendar** y **Google Drive/Slides** como integraciones reales de salida y de soporte operativo. El cálculo de distancias y la geocodificación de direcciones se resuelven con **OpenRouteService**.

Las secciones siguientes documentan, en orden: el mapa de arquitectura completo del sistema, la estructura de datos que funciona como memoria del ecosistema, la estrategia seguida para decidir dónde usar inteligencia artificial y dónde no, y la malla de seguridad, privacidad y resiliencia que protege el proceso ante datos faltantes, fallos técnicos y decisiones de alto riesgo.

Mapa de Arquitectura del Sistema

El ecosistema arranca con un único punto de entrada público: un formulario (Form Trigger de n8n) que carga el pedido de presupuesto directamente en la tabla `presupuestos` de Airtable, con estado inicial `Recibido`. A partir de ahí, una cadena de nodos determinísticos (Code, Switch, Airtable Search/Update) resuelve, sin intervención manual, la lista de precios aplicable según el cliente, geocodifica y valida las direcciones de origen y destino contra la API de OpenRouteService, calcula los kilómetros de traslado, resuelve el precio de rigging y el vehículo —y el acoplado, si el equipo lo requiere— disponible consultando el calendario compartido de flota en Google Calendar, calcula el flete según la distancia y la lista de precios, y suma el valor total con sus monedas ya validadas.

Cuando el proceso encuentra un caso que depende de criterio humano —necesidad de contratar terceros o de una visita de acceso al lugar de trabajo— el flujo se detiene explícitamente y avisa por Slack. En total quedan seis estados de excepción de negocio (`Datos incompletos`, `Revisar direcciones`, `Pendiente cotización manual`, `Sin vehículo disponible`, `Revisión de moneda`, `Esperando terceros` / `Esperando visita`), que un único trigger de reintento —con sondeo cada un minuto sobre el campo de última modificación del registro— retoma, cada uno desde el punto exacto de la cadena donde se cortó, apenas una persona carga la decisión correspondiente en Airtable.

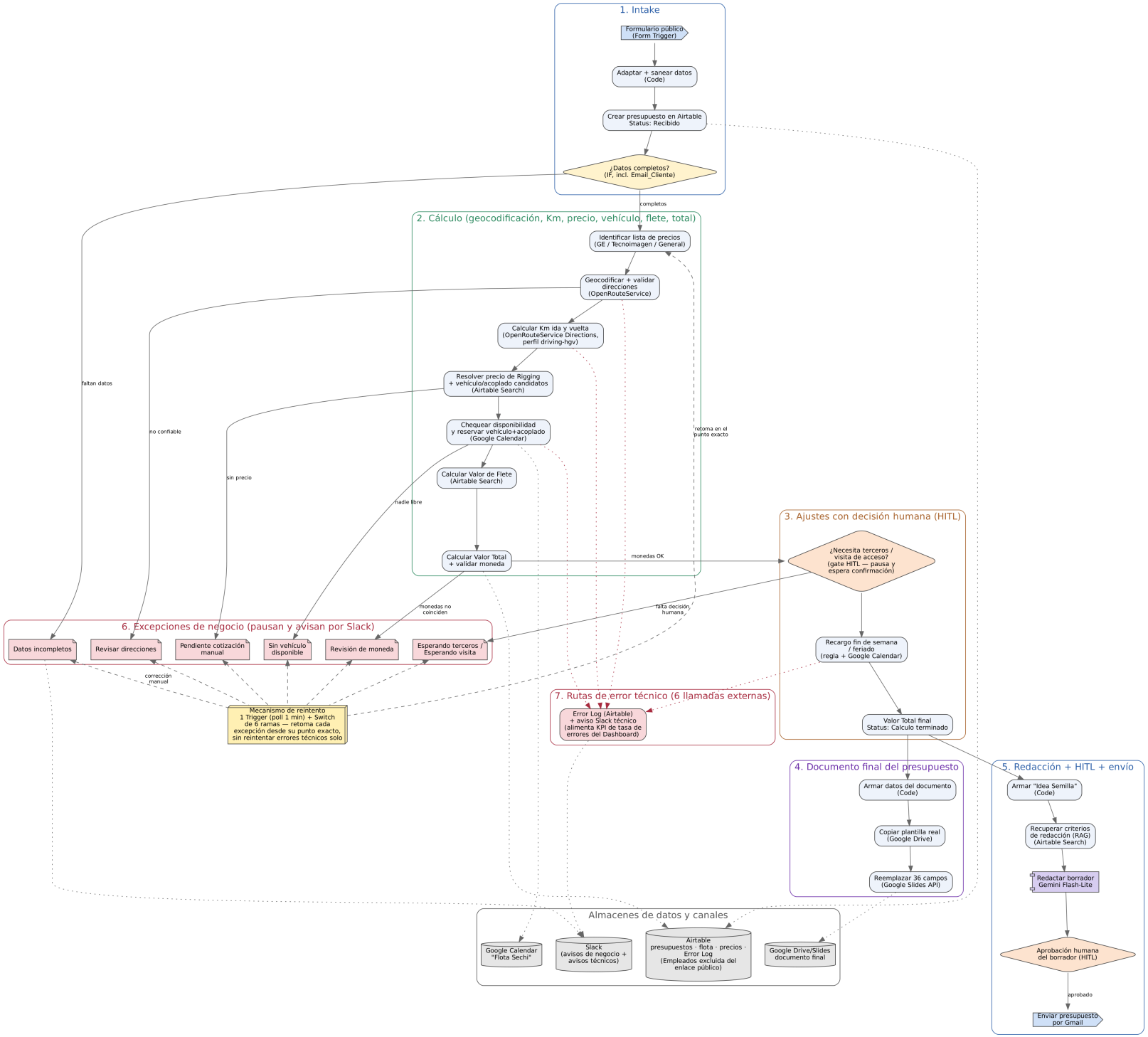
Una vez resuelto el cálculo completo —incluyendo el recargo del 10% por fin de semana o feriado, verificado contra el calendario oficial de feriados y contra el calendario de flota— el presupuesto sigue dos caminos en paralelo: la generación del documento final (una copia de la plantilla real de la empresa en Google Slides, con 36 de sus 37 campos editables completados automáticamente) y la redacción del cuerpo del email de presupuesto. Es en este último paso —y únicamente en este paso de todo el ecosistema— donde interviene un modelo de lenguaje: Gemini Flash-Lite recupera por RAG los criterios de redacción reales desde una base de conocimiento en Airtable y los combina con los datos ya calculados del caso, con instrucciones explícitas de no inventar ningún valor que no esté en esos datos. El borrador que redacta la IA nunca sale solo: una persona tiene que aprobarlo en Airtable antes de que el nodo de envío real por Gmail lo mande al cliente.

En paralelo a toda la cadena, los seis puntos del ecosistema que dependen de una API externa —las tres llamadas a OpenRouteService y las tres a Google Calendar— tienen manejo de error explícito: un fallo no interrumpe el workflow ni lo deja en un estado inconsistente. Se registra en una tabla dedicada (`Error Log`, con el nodo que falló, el mensaje real de la API y la fecha), dispara un aviso por un canal de Slack separado de los avisos de negocio, y queda esperando revisión humana antes de cualquier reintento: el sistema nunca reintenta solo un error técnico.

La página siguiente muestra el diagrama completo del ecosistema: los triggers de entrada, los puntos de ruteo condicional, las integraciones externas, el único nodo de inteligencia artificial, los puntos de decisión humana, el mecanismo de reintento y el destino final de los datos.

— El diagrama completo del ecosistema se muestra en la página siguiente —

Ecosistema de Automatización IA Autónomo — Transporte Sechi
Elaboración de Presupuestos de Rigging de Equipos Médicos



Manual Operativo de Estructuras de Datos

Base de Airtable — "Sechi — Redacción de Presupuestos"

Tabla central: `presupuestos`

Es el registro de cada caso, desde que entra por el formulario hasta que el presupuesto se envía por email.

Campos de intake (cargados por el formulario): `equipo`, `modelo`, `cantidad_bultos`, `cliente`, `direccion_origen`, `direccion_destino`, `fecha_propuesta`, `fotos_acceso`, `Tipo de trabajo`, `Variante` (Nuevo/Usado), `Nivel de Dificultad`, `Email_Cliente`.

Campos de cálculo (escritos por la cadena de cálculo): `Lista de precios aplicada`, `Km calculados`, `Vehiculo asignado` (vínculo a `flota`), `Valor de Rigging` + `Moneda Rigging`, `Valor de Flete` + `Moneda flete`, `Valor Total`, `Necesita Terceros` + `Valor de Terceros` + `Moneda Terceros` + `Concepto de Terceros`, `Valor Dia de Trabajo`, `Valor Desconsolidado`, `Valor Montaje Extendido`, `Camion de apoyo aplicado` + `Valor Camion de apoyo`, `Necesita Visita`, `Nº de Presupuesto` (autonumérico).

Campos de handoff y redacción: `Idea Semilla` (texto que resume el caso para el agente de redacción), `borrador generado`, `aprobado` (checkbox de aprobación humana).

Campo lookup: `Patente (from flota)` trae la patente real del vehículo vinculado en `Vehiculo asignado` — un campo de vínculo de Airtable solo expone, vía API, el identificador interno del registro vinculado, no el texto visible en la grilla, por lo que el lookup es necesario para cualquier integración que necesite mostrar la patente real (por ejemplo, el documento final del presupuesto).

Valores de `Status`, en el orden en que aparecen en el flujo: `Recibido` → `Datos incompletos` | `Revisar direcciones` | `Pendiente cotizacion manual` | `Sin vehiculo disponible` | `Esperando terceros` | `Esperando visita` | `Revision de moneda` → `Calculo terminado` → `generando` → `en revision` → `aprobado` | `rechazado` → `publicado`. Además, seis valores de reintento permiten retomar cada excepción desde su punto exacto: `Reintentar datos`, `Reintentar direcciones`, `Reintentar vehiculo`, `Reintentar moneda`, `Reintentar terceros`, `Reintentar cotizacion manual`.

Tablas de precio y flota

Tabla	Vinculada a	Particularidad
`flota`	—	9 registros (8 vehículos + 1 acoplado); `Patente` como campo primario
`precios_general` / `flete_y_varios_general`	`flota`	`Equipo` se combina con `Modelo` solo para Resonador
`precios_tecnoimagen` / `flete_y_varios_tecnoimagen`	`flota`	`Equipo` + `Modelo` para Resonador y Mamógrafo
`precios_ge_dificultad` / `Flete_y_terceros_ge`	`flota`	El precio de rigging está partido por nivel de dificultad en 6 columnas (`Dificultad_0_USD` a `Dificultad_5_USD`); `Equipo` nunca se combina con `Modelo`
`Niveles de dificultad - GE`	—	Tabla de referencia, 6 niveles (0 a 5)
`Empleados`	—	Datos personales de 21 personas reales — excluida del enlace público de solo lectura
`Error Log`	`presupuestos`	`Presupuesto` (vínculo), `Nodo`, `Mensaje de Error`, `Fecha y Hora` (automático), `Resuelto` (checkbox manual)

Los nombres de los campos de precio y concepto de flete, y el campo que trae las patentes candidatas en las tablas de precio, están unificados entre las tres listas de precio del ecosistema (General, Tecnoimagen y GE), para que la misma lógica de búsqueda funcione sin importar qué lista corresponda a cada caso.

Esquemas JSON de las integraciones

Entrada — formulario público

12 campos reales:

```
Tipo de trabajo        (obligatorio: Movimiento interno / Rigging Estandar)
Equipo                 (obligatorio: 24 opciones — equipos y combinaciones equipo+modelo)
Modelo                 (texto libre, opcional)
Cantidad de bultos     (número, opcional)
Cliente                (texto, obligatorio)
Dirección de origen    (texto, obligatorio)
Dirección de destino   (texto, obligatorio)
Fecha propuesta de traslado (fecha, obligatorio)
Fotos/video de acceso (link) (texto, opcional)
Estado del equipo      (opcional: Nuevo / Usado)
Nivel de dificultad    (texto, opcional)
Email del cliente      (obligatorio)
```

El campo "Nivel de dificultad" se armó deliberadamente como texto libre y no como número: con un campo numérico, un valor vacío en el formulario se interpreta como `0` —un nivel de dificultad numéricamente válido en sí mismo— y no hay forma de distinguir "no se cargó nada" de "se cargó un 0" explícito. Con texto libre, un campo vacío llega como string vacío real, y el nodo de normalización de datos puede aplicarle su valor por defecto sin ambigüedad.

Geocodificación — OpenRouteService

Pedido (`GET /geocode/search`, uno por cada dirección):

```
?api_key=<clave>&text=<dirección limpia>&size=1&boundary.country=AR
```

Respuesta relevante (formato GeoJSON — el orden de coordenadas es longitud/latitud):

```
{
  "features": [{
    "properties": { "housenumber": "...", "street": "...", "locality": "...", "region": "..." },
    "geometry": { "coordinates": [lon, lat] }
  }]
}
```

Cálculo de ruta — OpenRouteService Directions

Pedido (`POST /v2/directions/driving-hgv`, perfil de vehículo pesado):

```
{
  "coordinates": [[origen_lon, origen_lat], [destino_lon, destino_lat]],
  "units": "km"
}
```

Respuesta relevante: la distancia total de la ruta, ya expresada en kilómetros.

Calendario de flota — Google Calendar

Las reservas y los chequeos de disponibilidad de vehículo se identifican por la patente incluida en el título de cada evento (por ejemplo, `[MFK 183] - Cliente General - Resonador`). Los eventos de ocupación son de tipo "evento de todo el día", con fecha de fin exclusiva (un día después del último día real cubierto por el trabajo).

Documento final — Google Slides

El workflow usa la operación nativa de n8n para reemplazo de texto en una presentación de Google Slides: recibe el identificador de la presentación copiada desde la plantilla real de la empresa, y una lista de 36 pares de marcador de posición y valor real, uno por cada campo editable de la plantilla (`Hoja Presupuesto Nueva`, 37 cuadros editables en total: 36 automáticos y 1 de uso manual para observaciones).

Diagrama de relaciones entre tablas

```
flota ◄──────┬──────── precios_general.vehiculo
             ├──────── precios_tecnoimagen.vehiculo
             ├──────── precios_ge_dificultad.vehiculo
             ├──────── flete_y_varios_general.vehiculo
             ├──────── flete_y_varios_tecnoimagen.vehiculo
             ├──────── Flete_y_terceros_ge.vehiculo
             └──────── presupuestos.Vehiculo asignado  (+ lookup "Patente (from flota)")

presupuestos ◄──────── Error Log.Presupuesto

Empleados  (tabla aislada, sin vínculo a flota ni a presupuestos — asignación de cuadrilla manual)
```

Estrategia de Optimización de Costos y Recursos de IA

Principio de diseño: la IA se usa donde agrega valor, no por defecto

Del flujo completo de Elaboración de Presupuestos —intake, geocodificación, cálculo de kilómetros, identificación de lista de precios, resolución de vehículo y acoplado, cálculo de rigging/flete/total, chequeo de necesidad de terceros, recargo por fin de semana o feriado, reservas de calendario, generación del documento final y redacción del email— **un solo punto del ecosistema usa un modelo de lenguaje**. Todo lo demás se resuelve con lógica determinística (nodos de código, fórmulas de Airtable, condicionales) o con intervención humana explícita cuando el dato depende de criterio humano.

La tabla siguiente recorre cada tarea del pipeline y documenta qué la procesa y por qué.

Tarea	Qué la procesa	Por qué (no) usa IA
Clasificar tipo de trabajo	Campo declarado por el usuario en el formulario	El dato ya es un input directo del cliente; inferirlo con una IA leyendo texto libre gastaría recursos en algo que no reduce ninguna incertidumbre real.
Identificar lista de precios aplicable	Nodo de código, comparación exacta de dos strings	Regla de negocio binaria y estable (solo dos clientes tienen lista propia) — una IA no mejora la precisión de una comparación de igualdad.
Geocodificación y validación de direcciones	OpenRouteService + validación cruzada por código	El riesgo no es de lenguaje natural sino de confiabilidad de datos geográficos: se resuelve con validación de reglas, no con el razonamiento de un modelo de lenguaje, que además podría "alucinar" una coincidencia en vez de señalar la incertidumbre.
Cálculo de kilómetros, vehículo/acoplado, precios de rigging/flete/total	Nodos de código + búsquedas estructuradas en Airtable	Aritmética y búsquedas sobre datos ya tabulados — determinístico por naturaleza, sin ninguna ambigüedad que justifique un modelo probabilístico.
Necesidad de contratar terceros / categoría de riesgo del equipo	Pausa y confirmación humana (HITL)	Depende de juicio humano sobre fotos y contexto operativo real — ni un modelo de visión ni uno de texto tiene el contexto operativo de Sechi para decidir esto con la confiabilidad que exige un presupuesto real.
Recargo por fin de semana o feriado	Nodo de código + lista oficial de feriados	Regla de calendario, verificable contra una fuente primaria — no hay nada ambiguo que requiera lenguaje natural.
Documento final del presupuesto	Nodo de código + reemplazo de texto en Google Slides	Es un llenado de plantilla campo por campo, una sustitución de texto exacto, no una generación de contenido nuevo.
Redacción del cuerpo del email de presupuesto	**Gemini Flash-Lite, con RAG sobre la base de conocimiento de criterios de redacción**	La única tarea genuinamente de lenguaje natural del ecosistema: combinar datos estructurados del caso con criterios de estilo y condiciones comerciales de la empresa, en un texto coherente.

Por qué Gemini Flash-Lite para la tarea de redacción

La tarea de redacción es de combinación de texto, no de razonamiento complejo: el prompt ya entrega al modelo los datos del caso ya resueltos y los criterios de contenido ya recuperados por RAG desde Airtable, con reglas explícitas de qué incluir y qué no —el prompt real instruye textualmente a no inventar ningún valor que no esté en esos datos. No hace falta razonamiento multi-paso ni conocimiento general del mundo, sino seguir instrucciones de forma literal y redactar con fluidez. Para ese perfil de tarea, el modelo más liviano de la familia rinde igual que uno más grande, a una fracción del costo.

Precios verificados (Google AI, tier estándar de pago, USD por 1.000.000 de tokens, fuente: ai.google.dev/pricing):

Modelo	Input / 1M tokens	Output / 1M tokens
Gemini Flash-Lite (familia del modelo usado)	USD 0,30	USD 2,50
Gemini Flash (modelo estándar, no-Lite, misma generación)	USD 0,75	USD 3,75

Estimación de ahorro por presupuesto redactado (a partir de un prompt real de orden de magnitud ~1.500 tokens de entrada —instrucciones, criterios de RAG e Idea Semilla— y ~500 tokens de salida):

- Con Flash-Lite: ≈ USD 0,0017 por presupuesto.
- Con Flash estándar: ≈ USD 0,003 por presupuesto.
- **Ahorro: ~43% por cada presupuesto redactado**, usando el modelo correcto para la complejidad real de la tarea, sin resignar calidad.

Volumen real de Sechi: ≈9,4 solicitudes completas por día (las que llegan a presupuesto enviado) × 5 días/semana ≈ 47 presupuestos/semana, o bien **≈204 presupuestos/mes** y **≈2.444 presupuestos/año**.

Ahorro mensual/anual de IA, con ese volumen:

	Flash-Lite (elegido)	Flash estándar	Ahorro
Por mes (≈204 presupuestos)	≈ USD 0,35	≈ USD 0,61	≈ USD 0,26 (~43%)
Por año (≈2.444 presupuestos)	≈ USD 4,16	≈ USD 7,34	≈ USD 3,18 (~43%)

El monto absoluto es chico —tiene sentido, porque la redacción de un email es una tarea liviana y el volumen de Sechi es bajo—. El punto central no es cuánto se ahorra eligiendo un modelo Flash-Lite en vez de uno estándar dentro de la misma familia (es marginal), sino lo que se explica a continuación: la decisión de diseño que realmente importa para el costo es que catorce de las aproximadamente quince tareas de decisión del ecosistema no llaman a ningún modelo de IA.

El ahorro real y más importante: evitar llamadas a IA, no solo abaratarlas

El punto central de esta estrategia no es "qué modelo de IA es más barato", sino que de las aproximadamente quince tareas de decisión y cálculo del ecosistema, catorce no usan IA en absoluto. Si el diseño hubiera usado un modelo de lenguaje para cada paso —clasificar tipo de trabajo, decidir lista de precios, validar direcciones, decidir necesidad de terceros, y así sucesivamente—, el costo por presupuesto sería un múltiplo de lo que es hoy, y además cada paso determinístico ganaría una fuente de error no determinístico (alucinación) donde hoy no existe. Reservar la IA para la única tarea que realmente la necesita es, en sí mismo, la optimización de costos más grande del ecosistema.

Seguridad, Privacidad y Resiliencia

Minimización de datos

La base de Airtable "Sechi — Redacción de Presupuestos" contiene una tabla, `Empleados` (nombre, rol, DNI y teléfono de 21 personas reales), que no tiene ninguna relación operativa directa con el cálculo de presupuestos: es información de personal, no de negocio. El enlace público de solo lectura que se comparte como parte de este entregable corresponde a una vista específica de Airtable que excluye esa tabla por completo, en lugar de compartir la base entera. Ninguna otra tabla del ecosistema contiene datos personales de terceros ajenos al presupuesto en curso —los datos del cliente (nombre de empresa, email, direcciones) son datos de negocio necesarios para el presupuesto, no datos sensibles de una persona física.

Las credenciales (claves de API de OpenRouteService y Gemini, OAuth2 de Google, tokens de Slack y Airtable) viven únicamente en el gestor de credenciales de n8n, nunca en un nodo, en un campo de Airtable ni en el código fuente de los nodos de código.

Puntos de control humano (HITL)

El ecosistema tiene varios puntos donde el flujo se detiene deliberadamente a esperar una decisión humana, en vez de asumir un valor por defecto:

- **Necesidad de contratar terceros o categoría de riesgo del equipo.** El sistema nunca decide esto solo: pausa el presupuesto, avisa por Slack con los datos del caso, y retoma automáticamente la cadena de cálculo recién cuando el equipo de logística carga la decisión en Airtable. Ante información insuficiente, el valor por defecto es siempre "escalar a una persona", nunca "asumir que no hace falta".
- **Necesidad de visita o revisión de acceso**, cuando faltan las fotos del lugar de trabajo — mismo patrón de pausa explícita.
- **Aprobación del borrador de presupuesto redactado por IA**, antes de que se envíe nada al cliente por email. El texto que genera el modelo de lenguaje nunca sale de Sechi sin que una persona lo revise y apruebe.
- **Revisión manual de direcciones no confiables** y de **discrepancias de moneda** entre el valor de rigging y el valor de flete — en ambos casos el sistema prefiere frenar y avisar antes que calcular con un dato dudoso.

Rutas de error técnico

Los seis puntos del ecosistema que dependen de una API externa —OpenRouteService, para geocodificación de origen y destino y cálculo de ruta, y Google Calendar, para consulta de feriados/eventos y reserva de vehículo— tienen manejo de error explícito, de forma que un fallo de la API no interrumpe silenciosamente el workflow ni lo deja en un estado inconsistente. Cada fallo se registra en una tabla dedicada, `Error Log` (presupuesto afectado, nodo que falló, mensaje de error real, fecha y hora, y un check `Resuelto` que marca el equipo tras confirmar un reintento exitoso), y dispara un aviso por Slack en un canal separado de los avisos de negocio.

Decisión de diseño deliberada: un error técnico no reintenta solo. El sistema registra y avisa, pero no cambia el estado del presupuesto ni reintenta automáticamente. La persona revisa la causa —¿la API está caída?, ¿la credencial venció?, ¿se llegó a un límite de uso?— y, si corresponde, cambia el estado a mano al valor de reintento que ya existe para ese tramo de la cadena, el mismo mecanismo que usan las excepciones de negocio. Un reintento automático e inmediato sobre un error que no es transitorio —por ejemplo, una credencial vencida— generaría llamados repetidos inútiles en cada ciclo del sondeo de reintentos; por eso el diseño evita explícitamente cualquier bucle de reintento sin intervención humana.

Esta propiedad se verificó directamente contra el workflow exportado, no solo por observación en pantalla: se analizó programáticamente el grafo de nodos y conexiones del archivo real del workflow (100 nodos). No existe ningún ciclo en el grafo de ejecución —ninguna rama del flujo puede reentrar sobre sí misma de forma automática— y no quedan nodos huérfanos ni ramas sueltas sin destino: los únicos nodos sin conexión entrante son los cinco triggers del ecosistema, como corresponde.

Validación de tipos de datos en los filtros

Se revisaron programáticamente las condiciones de los trece nodos de filtro (IF/Switch) del workflow. Los trece comparan tipos de forma estricta y correcta: texto contra texto (por ejemplo, lista de precios aplicada o estado del presupuesto) y booleano contra booleano (por ejemplo, validez de la geocodificación, disponibilidad de vehículo, coincidencia de monedas, o la aprobación del presupuesto), todos con el chequeo de tipos en modo estricto.

Durante la construcción y durante las pruebas de estrés del sistema se encontraron y corrigieron dos casos puntuales donde el tipo configurado en la condición no coincidía con el tipo real del dato: uno donde un campo booleano (la aprobación del presupuesto) tenía activada una opción de conversión de tipos "laxa" a nivel de nodo, que relajaba el chequeo aunque el operador elegido ya fuera el correcto; y otro donde un campo numérico (el precio de rigging ya resuelto) estaba comparado con el tipo de dato "texto", lo que funcionaba mientras el campo estaba vacío pero producía un error de ejecución en cuanto llegaba un valor numérico real. Ambos se corrigieron ajustando el tipo de la condición al tipo real del dato, y quedaron documentados como clase de error recurrente a vigilar en cualquier nodo de filtro nuevo.

Mecanismo de reintento de excepciones de negocio

Las excepciones de negocio (datos incompletos, direcciones a revisar, falta de vehículo disponible, revisión de moneda, reintento de terceros, reintento de cotización manual) se retoman con un único trigger de Airtable —con sondeo cada un minuto sobre el campo de última modificación— y un enrutador de seis ramas, en vez de un trigger separado por cada excepción: esto evita que el mismo cambio dispare el workflow más de una vez. Cada rama retoma la cadena desde el punto exacto donde se cortó, sin repetir cálculos ni duplicar reservas de vehículo ya hechas, con un chequeo de idempotencia explícito en el gate de necesidad de terceros: si el dato ya está resuelto, una recalculación posterior no vuelve a pausar ni duplica un monto ya sumado.

Lecciones de resiliencia encontradas durante la construcción

Durante la construcción y las pruebas del ecosistema se encontraron y corrigieron, con evidencia verificada contra el archivo real del workflow —no solo por confirmación en pantalla— los siguientes patrones de falla. Se documentan porque cada uno es una clase de error que puede reaparecer en cualquier automatización similar, no un error puntual sin valor general:

- Un nodo que llama a una API (por ejemplo, una actualización o creación en Airtable, o la creación de un evento en Google Calendar) reemplaza su salida por la respuesta cruda de la API, no por una combinación con lo que entró: cualquier nodo posterior que dependa de un dato calculado antes tiene que referenciarlo explícitamente por nombre de nodo, nunca asumir que sigue disponible en la salida inmediata anterior.
- Conectar una rama de reintento directo a un nodo intermedio puede romper la ejecución si un nodo posterior referencia, por nombre, a un nodo anterior que esa rama nunca llega a ejecutar. La corrección sistemática fue reconectar siempre al mismo punto de entrada que un reintento ya probado y análogo.
- Espacios en blanco invisibles en expresiones de condición y en opciones de campos de selección única de Airtable son invisibles en la interfaz normal, pero rompen comparaciones de igualdad exacta sin ningún error explícito.
- Un campo de tipo "vínculo a otro registro" de Airtable devuelve, vía API, el identificador interno del registro vinculado, no el texto visible en la grilla, y devuelve un arreglo incluso cuando el vínculo es único. Se resolvió con un campo de tipo lookup dedicado y manejo explícito del arreglo.
- Un string vacío no es lo mismo que "sin valor" para un campo de selección única de Airtable: Airtable acepta un valor nulo para dejar el campo sin valor, pero rechaza un string vacío con un error explícito. Se corrigió asegurando que el código mande un valor nulo cuando no corresponde ningún valor, en vez de un string vacío.
- Una fórmula de búsqueda de Airtable que compara texto contra una opción de selección única es sensible a mayúsculas y minúsculas: una búsqueda de flete devolvía cero resultados para un caso con un equipo y un kilometraje perfectamente válidos, sin ningún error —simplemente "no encontraba nada", lo cual es más difícil de notar que un error explícito. Se corrigió unificando el texto de la opción en Airtable, para no romper las otras listas de precios que ya usaban la capitalización correcta.

- Un error 404 genérico de una API de Google puede tener causas completamente distintas que lucen igual de plausibles entre sí —permisos insuficientes, referencias a credenciales eliminadas, demoras de indexado entre servicios—. La causa real en este caso resultó ser un carácter de expresión escrito de más a mano en un campo que ya estaba en modo expresión, produciendo un identificador duplicado e inválido. La lección aplicada: ante un error genérico que persiste después de corregir una causa plausible, conviene aislar variables con una prueba controlada en vez de agregar soluciones defensivas que pueden parecer funcionar por coincidencia de timing, sin resolver la causa de fondo.

Verificación de seguridad contra el workflow real

El ecosistema no tiene ningún filtro que pueda entrar en un bucle infinito: el grafo de nodos no tiene ciclos —verificado programáticamente contra el archivo exportado del workflow— y el mecanismo de reintento de excepciones nunca reintenta automáticamente un error técnico, lo que elimina el escenario de un reintento en bucle sobre un error no transitorio. Los trece nodos de condición del workflow comparan tipos de datos correctos y en modo estricto, incluyendo los dos casos puntuales corregidos durante la construcción y documentados más arriba. Y el único prompt de inteligencia artificial del ecosistema es dinámico: no contiene texto de negocio fijo, inyecta los datos reales del caso puntual y los criterios de redacción recuperados por RAG mediante variables de n8n, e instruye explícitamente al modelo a no inventar valores ausentes.

Recursos complementarios

El repositorio que acompaña este documento contiene el archivo técnico `.json` completo del workflow de n8n (100 nodos), las capturas de evidencia del Test de Estrés realizado sobre el workflow productivo, y los enlaces públicos de solo lectura a cada tabla de la base de Airtable (excluyendo, a propósito, la tabla con datos personales del personal):

Repositorio: `github.com/emspomice-ai/sechi-ecosistema-ia-automatizacion`

Dashboard de Control Ejecutivo (vista pública de Airtable con los indicadores de volumen, tasa de aprobación/rechazo, escalados a cotización manual, tiempo promedio de aprobación y tasa de errores técnicos): `airtable.com/app2fFp8wPC2M1WCG/shrGyAzMoLCzZzkfB`

El resto de los enlaces de solo lectura a la base de datos (tabla central de presupuestos, base de conocimiento de redacción, flota, listas de precios y flete, y registro de errores técnicos) está listado en el `README.md` del repositorio.